

RTL2GDS中readme编写的问题

```
# 建议添加一行，一些小白可能在这个项目才开始用docker
docker run -it iedaopensource/iflow:latest
```

使用docker部署的问题

docker镜像中无法打开GUI

我使用mac连接ubuntu远程(没太解决明白这个远程图像显示到mac上)。
然后使用docker来作为运行环境，也面临一个宿主机访问远程生成图像的问题,希望也能在文档中提到该问题的解决方案。

```
root@a6b92f5c0a99:/opt/iFlow/scripts# Warning: could not connect to
display
Info: Could not load the Qt platform plugin "xcb" in "" even though it was
found.
root@a6b92f5c0a99:/opt/iFlow/scripts# ls
```

这个错误是因为在Docker容器中尝试运行图形界面程序（KLayout）时，无法连接到X11显示服务器。要解决这个问题，有几种方法：

1. 使用X11转发（推荐）：

```
# 在宿主机上允许X11转发
xhost +local:docker

# 启动Docker容器时添加X11相关参数
docker run -it \
    -e DISPLAY=$DISPLAY \
    -v /tmp/.X11-unix:/tmp/.X11-unix \
    iedaopensource/iflow:latest
```

2. 使用VNC：

```
# 在容器中安装VNC服务器
apt-get update
apt-get install -y x11vnc

# 启动VNC服务器
x11vnc -display :0
```

3. 将文件复制到宿主机查看：

```
# 在主机上执行
docker cp [容器
ID]:/opt/iFlow/result/uart.layout.klayout_0.26.2.sky130.HS.TYP.V1/uart.gds
./uart.gds

# 然后在主机上使用KLayout查看
klayout uart.gds
```

4. 使用卷挂载：

```
# 启动容器时挂载目录
docker run -it \
    -v /host/path:/opt/iFlow/result \
    iedaopensource/iflow:latest
```

5. 安装必要的X11依赖：

```
# 在容器中安装X11相关包
apt-get update
apt-get install -y \
    libxcb1 \
    libxcb-icccm4 \
    libxcb-image0 \
    libxcb-keysyms1 \
    libxcb-randr0 \
    libxcb-render-util0 \
    libxcb-shape0 \
    libxcb-sync1 \
    libxcb-xfixes0 \
    libxcb-xinerama0 \
    libxcb-xkb1
```

最佳实践建议：

1. 开发环境：

- 使用X11转发方式，这样可以直接在容器中运行图形界面程序
- 确保宿主机已安装X11服务器

2. 生产环境：

- 将文件复制到宿主机查看
- 使用VNC远程查看
- 使用Web界面查看

3. 文件管理：

- 使用卷挂载方式，方便文件共享

- 保持文件组织结构清晰
- 定期备份重要文件

4. 性能考虑：

- X11转发可能有一定性能开销
- 考虑使用VNC或Web界面查看大文件
- 注意网络带宽使用

5. 安全考虑：

- 限制X11转发范围
- 使用安全的传输方式
- 定期清理临时文件

这些方法可以根据具体需求选择使用。对于iFlow项目，推荐使用X11转发方式，或者将文件复制到宿主机查看。

太绕了，我选择直接源码部署,docker不但有性能损失还得手动复制gds文件。

我的平台

```
(base) along@alongforhappy: /home/along/Desktop/iFlow/scripts git:(master)
→ neofetch

      .-/+oosssso+/-.
      `:+ssssssssssssssssss+:`
    -+ssssssssssssssssssyyssss+-
    .ossssssssssssssssssdMMMNysssso.
Y9000K2020H
  /ssssssssssshdmmNNmyNMMMMhssssss/
+ssssssssshmydMMMMMMMMdddyssssssss+
/ssssssssshNMMMyhhyyyyhmNMMMNhssssssss/
(snap)
.ssssssssdMMMNhssssssssssshNMMMdssssssss.
+ssssshhyNMMNysssssssssssyNMMMyssssssss+
ossyNMMMNyMMhssssssssssssshmmhssssssso
ossyNMMMNyMMhssssssssssssshmmhssssssso
5.100GHz
+ssssshhyNMMNysssssssssssyNMMMyssssssss+
Corporation Device 1f55
.ssssssssdMMMNhssssssssssshNMMMdssssssss.
/ssssssssshNMMMyhhyyyhdNMMMNhssssssss/
+sssssssssdmydMMMMMMMMdddyssssssss+
/ssssssssssshdmNNNmyNMMMMhssssss/
.ossssssssssssssssssdMMMNysssso.
  -+ssssssssssssssssssyyssss+-
  `:+ssssssssssssssssss+:`

along@alongforhappy
-----
OS: Ubuntu 20.04.6 LTS x86_64
Host: 81YW Lenovo Legion

Kernel: 5.15.0-131-generic
Uptime: 7 hours, 16 mins
Packages: 3076 (dpkg), 12

Shell: zsh 5.8
Resolution: 1920x1080
Terminal: /dev/pts/0
CPU: Intel i7-10875H (16) @

GPU: NVIDIA 01:00.0 NVIDIA
GPU: Intel UHD Graphics
Memory: 3881MiB / 64002MiB
```

```
.-/+00SSSS00+/-.
```

iflow使用

使用配置脚本

ubuntu20.04 报错

```
→ ./run_flow.py -d aes_cipher_top -s synth

Current full name : aes_cipher_top.synth.yosys_0.9.sky130.HS.TYP.default
Previous full name : aes_cipher_top...sky130.HS.TYP.default

../../../../iFlow/tools/yosys4be891e8/bin/yosys: error while loading shared
libraries: libtcl8.5.so: cannot open shared object file: No such file or
directory
```

这个错误说明你在执行综合步骤时调用的 Yosys 依赖 libtcl8.5.so，而你的系统（Ubuntu 20.04）没有这个版本的动态库。Ubuntu 20.04 默认提供的是 libtcl8.6，所以你需要手动安装旧版本的库。

推荐解决方案：手动安装 libtcl8.5

步骤 1：下载 .deb 包

wget http://security.ubuntu.com/ubuntu/pool/universe/t/tcl8.5/libtcl8.5_8.5.19-4_amd64.deb

步骤 2：安装包

```
sudo dpkg -i libtcl8.5_8.5.19-4_amd64.deb
```

步骤 3：如果有依赖缺失，修复它们

```
sudo apt --fix-broken install
```

验证是否成功

```
ls /usr/lib/x86_64-linux-gnu/libtcl8.5.so
```

然后重新运行你的 iFlow 命令：

```
./run_flow.py -d aes_cipher_top -s synth
```

📦 可选方案（如果系统无法装 libtcl8.5）

如果遇到冲突或不能安装 .deb 包，你也可以：

- 将 libtcl8.5.so 放到 iFlow/tools/yosys 的 lib 目录下
- 设置环境变量：

```
export LD_LIBRARY_PATH=/path/to/libtcl8.5:$LD_LIBRARY_PATH
```

uart示例运行

```
uart = Flow('uart', 'asap7', 'HS', 'TYP', 'TYP')
```

1. 首先，这个文件定义了一个名为 `Flow` 的类（从导入语句 `from data_def import *` 可以看出），用于配置不同的设计流程。
2. 每个设计流程包含5个参数：
 - 设计名称（design name）
 - 工艺库（technology library）
 - 速度等级（speed grade）
 - 工艺角（process corner）
 - 额外参数（additional parameter）
3. 以 `ibex` 为例：

```
ibex = Flow('ibex_core', 'sky130', 'HS', 'TYP', '')
```

这表示：

- 设计名称： `ibex_core` （一个RISC-V处理器核心）
 - 工艺库： `sky130` （SkyWater 130nm工艺）
 - 速度等级： `HS` （High Speed）
 - 工艺角： `TYP` （Typical）
 - 额外参数： 空
4. 其他设计包括：
- `asic_top`：使用SMIC 110nm工艺
 - `aes`：使用SkyWater 130nm工艺的AES加密模块
 - `gcd`：使用SkyWater 130nm工艺的GCD模块
 - `uart`：使用ASAP7 7nm工艺的UART模块
 - `picorv32`：使用SkyWater 130nm工艺的PicoRV32处理器

这个配置文件主要用于设置不同数字电路设计项目的工艺参数，这些参数将用于后续的物理设计流程，如综合、布局布线等。每个设计都根据其特定需求选择了合适的工艺库和参数配置。

UART设计的流程：

1. 设计概述

- UART（通用异步收发器）是一个百门级到千门级的数字设计
- 用于异步通信的串行数据总线设计

2. 准备工作

- 首先需要将UART的RTL代码放在 `iFlow/rtl` 目录下
- 在 `iFlow/scripts/cfg/flow_cfg.py` 中定义UART设计的流程参数
- 从配置文件中可以看到，UART设计最初配置为使用ASAP7工艺库，但也可以使用sky130工艺库

3. 脚本准备

- 进入 `iFlow/scripts` 目录
- 使用命令 `cp -r gcd uart` 复制GCD设计的脚本作为UART设计的基础
- 需要修改的主要是综合脚本 `synth.yosys_0.9.tcl`，将输入文件改为UART的RTL代码

4. 布局规划

- 由于UART设计与GCD设计规模相近，可以沿用GCD的floorplan设置
- 可以根据需要调整DIE_AREA（芯片面积）和CORE_AREA（核心区域）

5. 运行流程

使用以下命令运行完整的后端流程：

```
./run_flow.py -d uart -s
synth,floorplan,tapcell,pdn,gplace,resize,dplace,cts,filler,groute,droute,
layout -f sky130 -t HS -c TYP -v V1 -l V1
```

这个命令包含了以下步骤：

- `synth`：综合
- `floorplan`：布局规划
- `tapcell`：添加tap单元
- `pdn`：电源网络设计
- `gplace`：全局布局
- `resize`：单元大小调整
- `dplace`：详细布局
- `cts`：时钟树综合

- `filler`: 填充单元
- `groute`: 全局布线
- `droute`: 详细布线
- `layout`: 生成最终版图

参数说明:

- `-d uart`: 指定设计名称为uart
- `-f sky130`: 使用sky130工艺库
- `-t HS`: 使用高速工艺角
- `-c TYP`: 使用典型工艺角
- `-v V1 -l V1`: 版本和标签设置

这个流程是一个完整的数字后端设计流程，从RTL代码开始，经过综合、布局、布线等步骤，最终生成可以用于制造的版图。

RTL2GDS体验流程

完全按照流程走

会一个一个爆错，提示缺失module

```
Traceback (most recent call last):
  File "/usr/lib/python3.10/runpy.py", line 196, in _run_module_as_main
    return _run_code(code, main_globals, None,
  File "/usr/lib/python3.10/runpy.py", line 86, in _run_code
    exec(code, run_globals)
  File "/rtl2gds/src/rtl2gds/__main__.py", line 47, in <module>
    main()
  File "/rtl2gds/src/rtl2gds/__main__.py", line 39, in main
    flow.rtl2gds_flow.run(chip_design)
  File "/rtl2gds/src/rtl2gds/flow/rtl2gds_flow.py", line 20, in run
    runner.run_floorplan()
  File "/rtl2gds/src/rtl2gds/flow/step_wrapper.py", line 73, in
run_floorplan
    metrics, artifacts = step.floorplan.run(
  File "/rtl2gds/src/rtl2gds/step/floorplan.py", line 69, in run
    raise subprocess.CalledProcessError(ret_code, shell_cmd)
subprocess.CalledProcessError: Command '['iEDA', '-script',
'/rtl2gds/tools/iEDA/script/iFP_script/run_iFP.tcl']' returned non-zero
exit status 127.
```

单独运行

```
root@7a627ed631f6:/rtl2gds/design_zoo/picorv32a# iEDA -script
/rtl2gds/tools/iEDA/script/iFP_script/run_iFP.tcl
iEDA: error while loading shared libraries: libglog.so.0: cannot open
shared object file: No such file or directory
```

docker部署设置的环境变量大有问题。包安装的有问题。待整理待提交若干PR

tmux的妙用

使用mac连接服务器跑任务时，会遇到这样的情况，担心SSH连接断掉导致服务器中进程也被中止。

要实现 SSH 断开后程序还能继续运行，最推荐的方法就是使用 tmux，它可以在后台维持终端会话，哪怕你退出 SSH，程序也不会中断。

下面是完整教程：

✅ 一次性设置流程（适用于任何程序）

第 1 步：连接远程服务器

```
ssh yourname@your-server-ip
```

第 2 步：创建 tmux 会话

```
tmux new -s mysession
```

这会新建一个名为 mysession 的会话并进入其中。

第 3 步：在 tmux 中运行你的程序

比如你要运行 iFlow：

```
./run_flow.py -d picorv32 -s droute
```

第 4 步：退出 tmux（但不终止程序）

按下：

```
Ctrl + b 然后松开，再按 d
```

你会看到提示：[detached (from session mysession)]

这表示你安全退出了 tmux，会话还在后台运行。

第 5 步：下次 SSH 再连接回来，重进会话

```
ssh yourname@your-server-ip
```



```
tmux attach -t mysession
```

🔑 查看/管理多个会话（如果你忘了名字）

```
tmux ls
```

❌ 关闭会话（程序跑完以后）

进入会话后按：

```
exit
```

或手动 **Ctrl + C** 停止程序，tmux 会自动关闭空会话。

🔄 总结口诀

```
tmux new -s 名字      # 创建并进入
Ctrl+b, d            # 暂时离开
tmux attach -t 名字   # 回来继续
tmux ls               # 看有哪些会话
```

如果你用的是 VS Code Remote SSH 模式，VS Code 断掉会导致进程终止，而 tmux 能完美解决这个问题。

首先运行uart设计熟悉iFlow流程

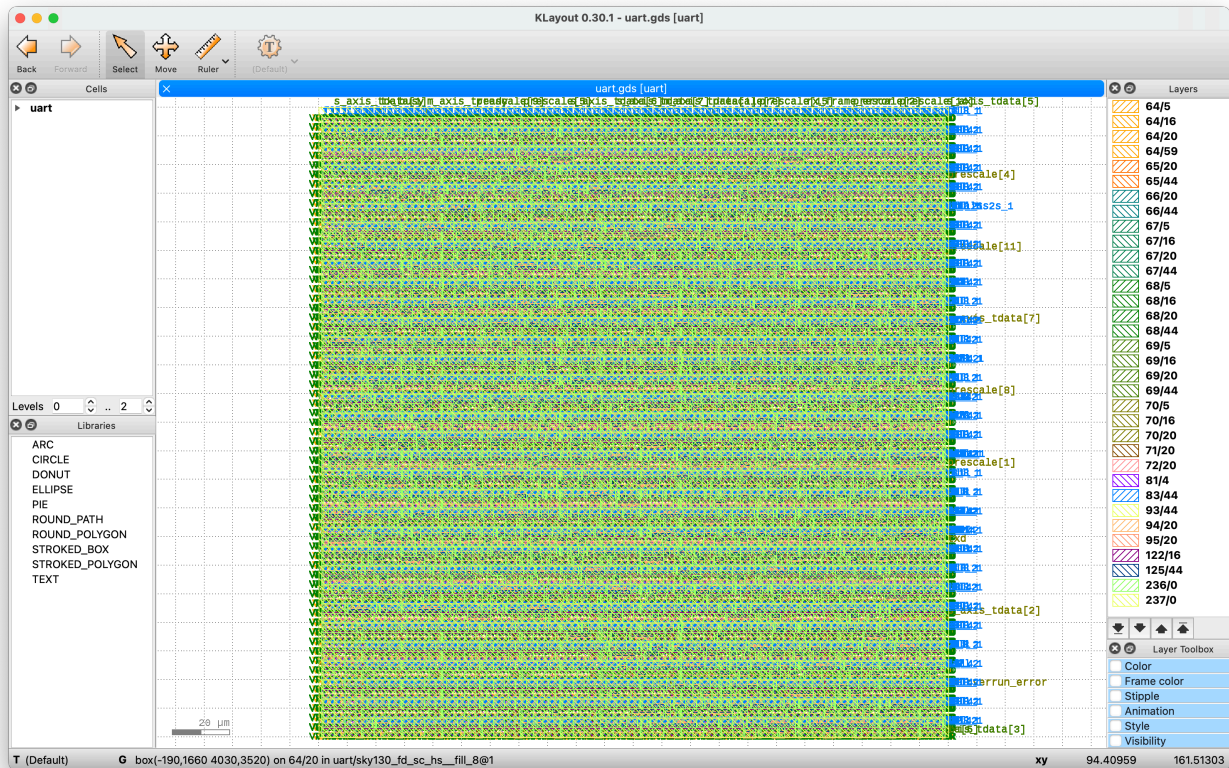
uart设计是一种通用串行数据总线的设计，是一个百门级到千门级的设计，用于异步通信。下面将讲述一下如何将iFlow中的design更换为uart设计。

首先，需要把uart相关的rtl代码放在“iFlow/rtl”目录下，如图34所示，然后要定义uart设计的flow，进入到“iFlow/scripts/cfg”目录下，编辑脚本“flow_cfg.py”，加入uart设计的默认flow参数，如图35所示，这里设置的默认foundry为“asap7”，改为“sky130”也是可以的。

其余的步骤可以不用修改，进入目录“iFlow/scripts”，运行命令：

```
./run_flow.py -d uart -s
synth,floorplan,tapcell,pdn,gplace,resize,dplace,cts,filler,groute,droute,
layout -f sky130 -t HS -c TYP -v V1 -l V1
```

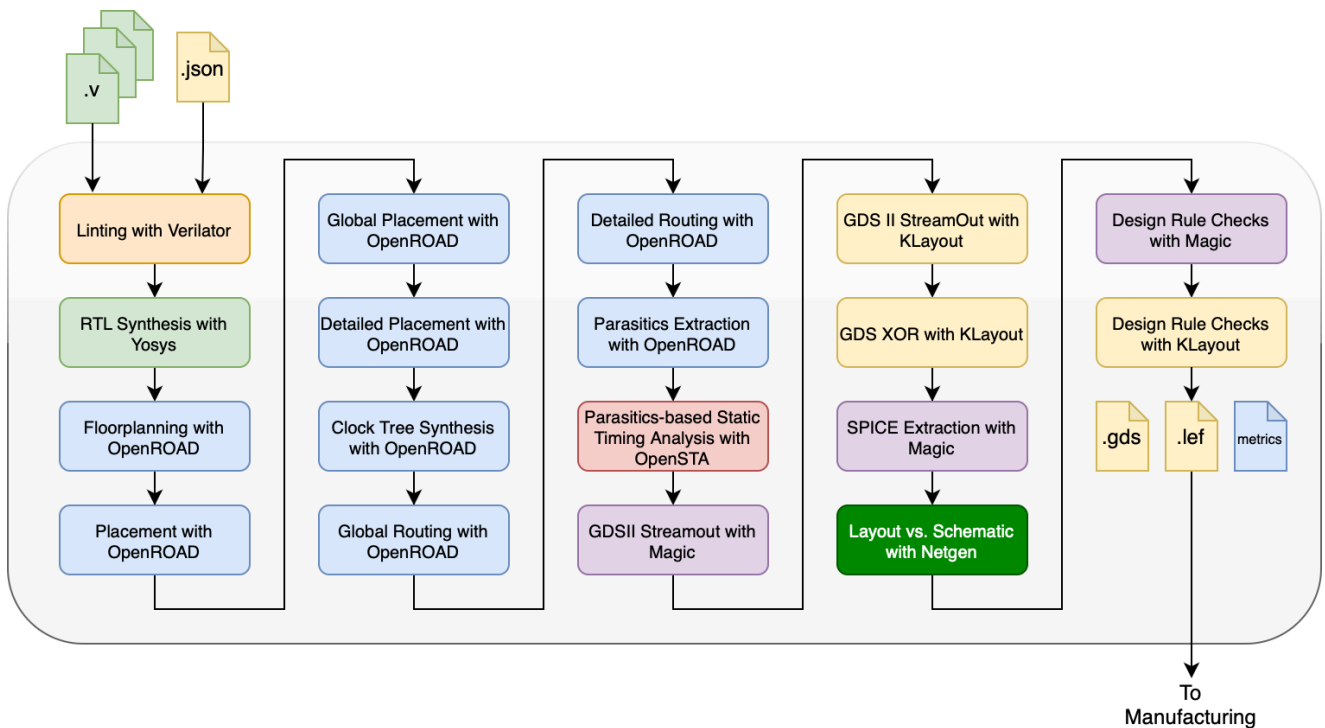
即可基于sky130工艺跑uart设计的后端流程，结果如图所示。



iFlow的文档看的一头雾水，我将先参考一下OpenLane的文档跑出来picorv32先。

OpenLane

OpenLane架构



大概了解一下RTL到GDS的过程中需要注意的地方。

iFlow体验

先卡在了布局阶段

```
ABC: ScL_LibertyReadGenlib() skipped three-state cell
"sky130_fd_sc_hs__ebufn_2".
ABC: ScL_LibertyReadGenlib() skipped three-state cell
"sky130_fd_sc_hs__ebufn_4".
ABC: ScL_LibertyReadGenlib() skipped three-state cell
"sky130_fd_sc_hs__ebufn_8".
ABC: ScL_LibertyReadGenlib() skipped sequential cell
"sky130_fd_sc_hs__edfxbp_1".
ABC: ScL_LibertyReadGenlib() skipped sequential cell
"sky130_fd_sc_hs__edfxtp_1".
ABC: ScL_LibertyReadGenlib() skipped three-state cell
"sky130_fd_sc_hs__einvn_1".
ABC: ScL_LibertyReadGenlib() skERROR: Can't open ABC output file
`/tmp/yosys-abc-KVSWxH/output.blif'.
```

在确认磁盘 **tmp** 空间正常后，继续尝试手动运行 **abc** 看报错，

尝试手动运行 **abc** 看报错

观察到你运行时 `/tmp/yosys-abc-*/` 目录下：

- 有 `abc.script`、`input.blif`、`stdcells.genlib`
- 但始终没有 `output.blif` 被生成
- 说明 ABC 在执行过程中根本没有成功生成输出文件

✅ 你现在的情况基本确认是：

ABC 执行失败，没有报出详细错误，但未生成 `output.blif`。

这通常是因为：

1. `.lib`（转换成 `.genlib`）中大部分逻辑单元不被 ABC 接受或不包含功能定义；
2. ABC 运行时遇到非法逻辑结构但不报错；
3. 环境变量或文件权限/路径问题（现在暂时不考虑这个，因为你能读写该目录）。

✅ 建议下一步这样处理：

🔧 方法一：手动运行 ABC 并开启 debug 模式

复制当前生成的文件夹进行调试：

```
cp -r /tmp/yosys-abc-KVSWxH ~/abc_debug
cd ~/abc_debug
~/Desktop/iFlow/tools/yosys4be891e8/bin/yosys-abc -s -f abc.script
```

如果能跑出来会生成 output.blif，否则就会直接在终端报错。

你还可以尝试：

```
~/Desktop/iFlow/tools/yosys4be891e8/bin/yosys-abc
```

错误信息中有关键点：

```
+ read_constr -v /home/along/Desktop/iFlow/rtl/picorv32/picorv32.sdc
Cannot open input file
"/home/along/Desktop/iFlow/rtl/picorv32/picorv32.sdc".
** cmd error: aborting 'source abc.script'
```

说明 ABC 在执行 abc.script 里的 read_constr 命令时，找不到时序约束文件 picorv32.sdc。

这就是导致 ABC 无法完成并报错的根本原因。

解决办法：

1. 确认文件是否存在

```
ls -l /home/along/Desktop/iFlow/rtl/picorv32/picorv32.sdc
```

- 如果不存在，说明路径错了或者文件没放好。
- 2. 如果文件不存在，你有该 SDC 文件吗？
- 如果没有，你需要从项目或者网络资源获取这个时序约束文件。
- 如果暂时不需要约束，可以临时修改 abc.script 去掉或注释掉 read_constr 这行。

3. 修改 abc.script

找到 read_constr -v /home/along/Desktop/iFlow/rtl/picorv32/picorv32.sdc 这一行，注释掉

4. 再重新运行 ABC

```
~/Desktop/iFlow/tools/yosys4be891e8/bin/yosys-abc -s -f /tmp/yosys-abc-KVSWxH/abc.script
```

或者你复制到本地目录后手动运行调试。

发现原来是我的sdc文件名后缀命名错误。同时单步执行也让我更容易定位问题。

[ERROR GPL-0301] Utilization exceeds 100%.

需要修改 `/iFlow/scripts/picorv32/floorplan.openroad_1.2.0.tcl`

```
#set DIE_AREA          "0 0 220.2 220.2"
#set CORE_AREA          "1.08 1.08 219.12 219.12"
set DIE_AREA            "0 0 2020.2 2020.2"
set CORE_AREA           "1.08 1.08 2019.12 2019.12"
```

1. Core Area定义错误

- 你日志里看到CoreAreaLxLy: 960 0和CoreAreaUxUy: 218880 216450, core面积计算很大 ($4.7e10 \text{ dbu}^2$), 确认这是否合理。

- Core定义如果不合理, 会导致单元布局范围混乱。

2. 实例数量和面积统计异常

- 总实例数14714, 放置实例14082, 面积累积巨大。

- 但是面积统计 (PlaceInstsArea: 176776646400) 比Core面积还大, 造成超利用率。

3. DEF或LEF文件是否匹配

- LEF中芯片尺寸、宏单元和标准单元尺寸需与DEF和设计匹配。

- 栅格和site尺寸是否与LEF定义一致。

4. Floorplan设置不合理

- floorplan脚本中的core区域定义, 是否误设成了太小的范围, 导致放置实例溢出。

- 重新确认floorplan配置, 确保core区域足够且合理。

5. PDN或tapcell阶段对布局没有限制或约束

- tapcell插入通常不会改变实例数, 但可能对布局边界有要求。

```
[INFO ODB-0225]      Created 1 library cells
[INFO ODB-0226] Finished LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_ef_io__vssio_lvc_pad.lef
[INFO ODB-0222] Reading LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_fd_io__top_xres4v2.lef
[INFO ODB-0225]      Created 1 library cells
[INFO ODB-0226] Finished LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_fd_io__top_xres4v2.lef
[INFO ODB-0222] Reading LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130io_fill.lef
[INFO ODB-0225]      Created 2 library cells
[INFO ODB-0226] Finished LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130io_fill.lef
[INFO ODB-0222] Reading LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_sram_1rw1r_128x256_8.lef
[INFO ODB-0221] 1000000 lines parsed!
[INFO ODB-0225]      Created 1 library cells
[INFO ODB-0226] Finished LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_sram_1rw1r_128x256_8.lef
```

```

ef
[INFO ODB-0222] Reading LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_sram_1rw1r_44x64_8.lef
[INFO ODB-0225] Created 1 library cells
[INFO ODB-0226] Finished LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_sram_1rw1r_44x64_8.lef
[INFO ODB-0222] Reading LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_sram_1rw1r_64x256_8.lef
f
[INFO ODB-0221] 1000000 lines parsed!
[INFO ODB-0225] Created 1 library cells
[INFO ODB-0226] Finished LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_sram_1rw1r_64x256_8.lef
f
[INFO ODB-0222] Reading LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_sram_1rw1r_80x64_8.lef
[INFO ODB-0225] Created 1 library cells
[INFO ODB-0226] Finished LEF file:
/home/along/Desktop/iFlow/foundry/sky130/lef/sky130_sram_1rw1r_80x64_8.lef
[INFO ODB-0127] Reading DEF file:
/home/along/Desktop/iFlow/result/picorv32.resize.openroad_1.2.0.sky130.HS.
TYP.default/picorv32.def
[INFO ODB-0128] Design: picorv32
[INFO ODB-0130] Created 409 pins.
[INFO ODB-0131] Created 59149 components and 281634 component-
terminals.
[INFO ODB-0132] Created 2 special nets and 236596 connections.
[INFO ODB-0133] Created 14264 nets and 45002 connections.
[INFO ODB-0134] Finished DEF file:
/home/along/Desktop/iFlow/result/picorv32.resize.openroad_1.2.0.sky130.HS.
TYP.default/picorv32.def
Placement Analysis
-----
total displacement      735617.1 u
average displacement    12.4 u
max displacement        371.7 u
original HPWL           932041.3 u
legalized HPWL          2120837.4 u
delta HPWL              128 %

```

从RTL到GDS

本试验最终采用 <https://github.com/YosysHQ/picorv32> 仓库中 `picorv32.v` 的作为rtl代码。

修改scripts/mycpu_top/synth.yosys_0.9.tcl

手动指明需要编译的文件，感觉很不方便啊,我相信有简易的文本处理函数能够完成该功能（当然有前提需要路径中没有无关的文件,也可以类似.gitignore这样的思路让用户把不需要参与编译的文件显示声明在.ifowignore中）

```
set VERILOG_INCLUDE_DIRS "\
"
set VERILOG_FILES " \
$RTL_PATH/picorv32.v \
```

修改core area和 die area

经过多次试验，首先是完全不知道要修改这两个参数。

在 floorplan 这一步中，主要对芯片进行面积和形状规划。通常情况下，chip-level的 floorplan由Core Area和Pad Area组成。Core Area用于摆放宏单元和标准单元。

在 Core Area 内插入 tapcell，tapcell 的作用是为所有标准单元的 N 阱和衬底提供偏置电源。在 另外还需要在边界处、sram 及 ip 周围插入 endcap，目的是消除不对称性，平衡密度。

PDN

全称power plan。这一步主要是构建为整个芯片供电的电源网络，电源网络质量会直接影响整体芯片的性能。

gplace

全称global place。在完成电源网络的构建后，接下来需要将标准单元摆放到 core area 范围中，在gplace 阶段，需要配置的主要参数有两个，一个为线 RC 参数的抽取层，主要是为了在 gplace 阶段抽取线 RC 参数进行延时的评估，从而更好地优化标准单元的摆放位置；另一个为布局密度，这一参数是用于设置摆放标准单元时的密度。

resize

在 dplace 前，进行一部分标准单元的更换及插入，其中包括将逻辑 0 和逻辑 1 的驱动端加上 Tie cell（电压钳位单元）和在需要 fix fanout（改善扇出）的驱动端加上buffer。

dplace

全称detail place。dplace 的主要作用是对 gplace 阶段已经摆放的标准单元进行合法化，消除标准单元之间的重叠，将标准单元对齐到 core area 范围内的 Row 上，从而确保电源网络能为标准单元供电。

所以可以说core area和die area决定了芯片后端全流程，当然可以，以下是更正式、书面化的版本，适合用于报告撰写：

芯片后端设计中的 Core Area 与 Die Area 概述

在芯片后端设计流程中，Core Area（核心区域）与 Die Area（芯片边界区域）是两个基本且至关重要的概念，它们分别定义了芯片中用于逻辑实现的核心区和整体的物理封装范围。

1. Die Area（芯片封装区域）

Die Area 是指整个芯片在晶圆上的矩形封装边界。它不仅包含了逻辑功能模块所在的 Core Area，还包括外围的 I/O 环、封装焊盘（pad ring）、电源环（power ring）、ESD 结构、测试结构、scribe line 等。Die Area 的大小直接影响芯片的封装成本与制造良率，因此需要在功能需求和工艺限制之间进行权衡。

2. Core Area（功能逻辑核心区域）

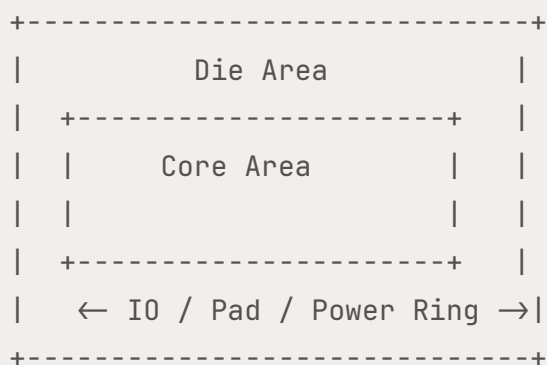
Core Area 是芯片中用于放置标准单元（standard cells）、宏单元（macros）及执行实际逻辑功能的区域。它位于 Die Area 内部，并预留出一定的边界距离，用于布置外围的 I/O 和电源结构。Core Area 的面积通常依据门级网表中单元的数量、布图密度（placement density）以及目标性能需求进行估算。

3. 二者关系

在实际设计中，设计人员通常根据逻辑电路规模估算 Core Area 的尺寸，并结合所需的外围资源（如 IO pad、ESD 电路等）确定 Die Area。两者的关系可表示为：

$\text{Die Area} = \text{Core Area} + \text{Core-to-Die Margin (外围边界)}$

如下图所示，Core Area 通常位于 Die 的中央部分：



4. 设计工具支持

在后端设计工具（如 Cadence Innovus、Synopsys ICC 或 iFlow 工具链）中，设计人员可以通过设置 core-to-die 距离或使用 floorplan 命令精确指定 Core Area 和 Die Area 的尺寸。合理的区域规划有助于提升布图效率、优化信号完整性，并满足工艺规则检查（DRC）与版图与原理图一致性检查（LVS）的要求。

在使用 iFlow 进行芯片后端设计时，util（Utilization，利用率）是一个非常关键的指标，它直接反映了 core area 内部逻辑单元的拥挤程度，对布局、布线、时序收敛、甚至能耗都有重

要影响。在你的报告中，可以这样进行书面化描述：

后端设计中的 Utilization（利用率）指标说明

在 iFlow 工具链中，util 指的是芯片核心区域（Core Area）中标准单元（Standard Cells）的占用率，通常以百分比形式表示：

$$\text{Utilization} = \frac{\text{Cell Area}}{\text{Core Area}} \times 100\%$$

其中：

- Cell Area 指所有标准单元总面积；
- Core Area 是标准单元可以被放置的实际区域。

利用率的作用与意义

- 低利用率（例如 < 50%）：虽然布图更容易，拥塞少，但面积利用效率低，芯片尺寸可能偏大，成本增加。
- 适中利用率（一般为 60%~80%）：是多数设计的目标范围，兼顾了布线空间和性能优化的可行性。
- 高利用率（例如 > 85%）：可能导致布线拥堵、时序难以收敛，甚至违反设计规则（如 DRC 违例），通常需要谨慎使用。

iFlow 中利用率的获取与调整

在运行 iFlow 的 floorplan 或 placement 阶段，工具通常会输出类似如下的信息：

```
Placement utilization: 72.34%
```

这个值反映的是当前布图密度，对于判断是否需要调整 core area 尺寸或重新进行 floorplan 十分关键。如果利用率过高，可通过以下方式优化：

- 增大 core area；
- 减小 cell density（在 floorplan 配置中调整）；
- 推迟 macro 的布局，预留更多空间给标准单元；
- 使用更高密度的 cell library（工艺允许的前提下）。

实际设计建议

设计早期应预估门数与目标频率，结合综合报告中的 cell 面积，合理分配 core area，目标是实现良好的 布局可行性、时序裕量 与 面积成本平衡。此外，后续物理综合（CTS、routing）过程中也要密切关注利用率变化，必要时进行增量 floorplan 调整。

多次调整

在sky130工艺库下，我多次调整area，首先是 2000*2000 左右的大小，时钟周期设置为420发现虽然violations在一段时间后陡然增加并且在 droute 这一步耗时非常久(虽然我的平台内存64G仍然跑了一夜没有出结果)

更换工艺库

sky130工艺金属层过少导致overflow难以解决。因此我更换了nangate45工艺，这一工艺拥有更多的金属层。更换之后overflow很快就被解决。

特性	Sky130	Nangate45	Nangate45 解决溢出问题的作用
工艺节点	130nm CMOS, 开源工艺, 适合低成本和研究用途	45nm CMOS, 商业化工艺, 适合高性能ASIC设计	更小特征尺寸减少单元和互连占用, 提升布线密度, 降低溢出风险
金属层数量	6层 (1层局部互连 + 5层金属 met1-met5)	8-10层金属 (视配置而定)	更多金属层提供额外布线资源, 优化信号和电源布线, 显著减少拥堵和溢出
设计规则	较宽松, 适合较大特征尺寸, 寄生效应较高	更严格, 适应短通道效应, 寄生效应较低	更密集的布线规则支持高密度布线, 减少布线通道竞争, 降低溢出
标准单元库	开源库 (如 sky130_fd_sc_hd), 单元尺寸较大, 布线需求高	高密度优化库, 单元尺寸小, 布线需求低	更小单元尺寸减少布线资源占用, 优化布局密度, 降低溢出风险
性能与功耗	工作电压 1.8V/3.3V/5V, 性能较低, 功耗较高	工作电压 1.0V-1.2V, 高性能, 低功耗	间接作用: 更优的性能和功耗允许更灵活的优化策略, 减少布线调整需求

- 溢出问题解决的核心原因: Nangate45 提供更多金属层 (8-10层 vs Sky130 的 6层) 和更小的特征尺寸 (45nm vs 130nm), 显著增加布线资源, 减少拥堵。

修改flow_cfg.py

```
asic_top      = Flow('asic_top', 'smic110', 'HD', 'MAX', 'iEDA')
aes           = Flow('aes_cipher_top', 'sky130', 'HS', 'TYP', '')
gcd           = Flow('gcd', 'sky130', 'HS', 'TYP', '')
uart          = Flow('uart', 'asap7', 'HS', 'TYP', '')
ibex          = Flow('ibex_core', 'sky130', 'HS', 'TYP', '')
picorv32      = Flow('picorv32', 'nangate45', 'HD', 'TYP', '')
```

我继续采用 2000*2000 的area大小, 发现过程中的violations能够得到降低, 但最终并不能收敛到0, 运行了七八个小时, 迭代58th只能收敛到206 violations, 不能得到最终结果, 于是继续降低标准。

虽然area越大布线越容易, 但是对于平台运算的时间和压力更大,

因此我根据在 2000*2000 area的各项指标, 采取将area缩减一半, utilization保持不变, 同时降低 density 从0.5到0.4, 降低主频将period修改为600

果然曾经运行一整夜都无法收敛, 再到运行五个小时只能迭代到较小数量的violations, 最后仅用时十来分钟就运行完毕。

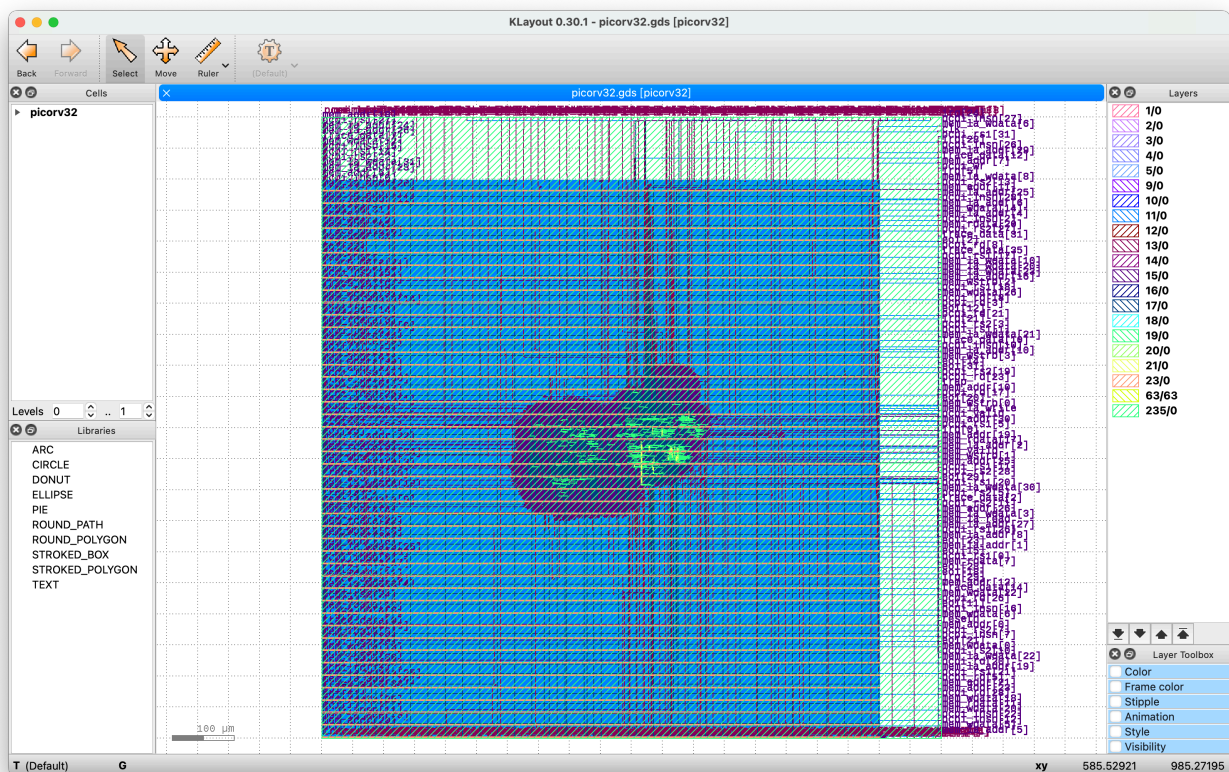
```

completing 10% with 0 violations
elapsed time = 00:00:46, memory = 3694.96 (MB)
completing 20% with 0 violations
elapsed time = 00:04:40, memory = 3732.22 (MB)
completing 30% with 75520 violations
elapsed time = 00:05:02, memory = 3726.93 (MB)

```

最终GDS

很快啊，啪的一下就出结果了



发现有效面积太小，开始多次尝试

```

set DIE_AREA "0 0 600.2 600.2"
set CORE_AREA "1.08 1.08 580.12 580.12"

```

```

[INFO DRT-0172] cpu time = 01:10:09, elapsed time = 00:10:45, memory =
1310.07 (MB), peak = 1310.07 (MB)

```

```

[INFO DRT-0180] post processing ...

```

```

Current full name :

```

```

picorv32.layout.klayout_0.26.2.nangate45.HD.TYP.default

```

```

Previous full name :

```

```
picorv32.drout.openroad_1.2.0.nangate45.HD.TYP.default
```

```
klayoutInsertLef.py : Insert lefs into lyt file of klayout
```

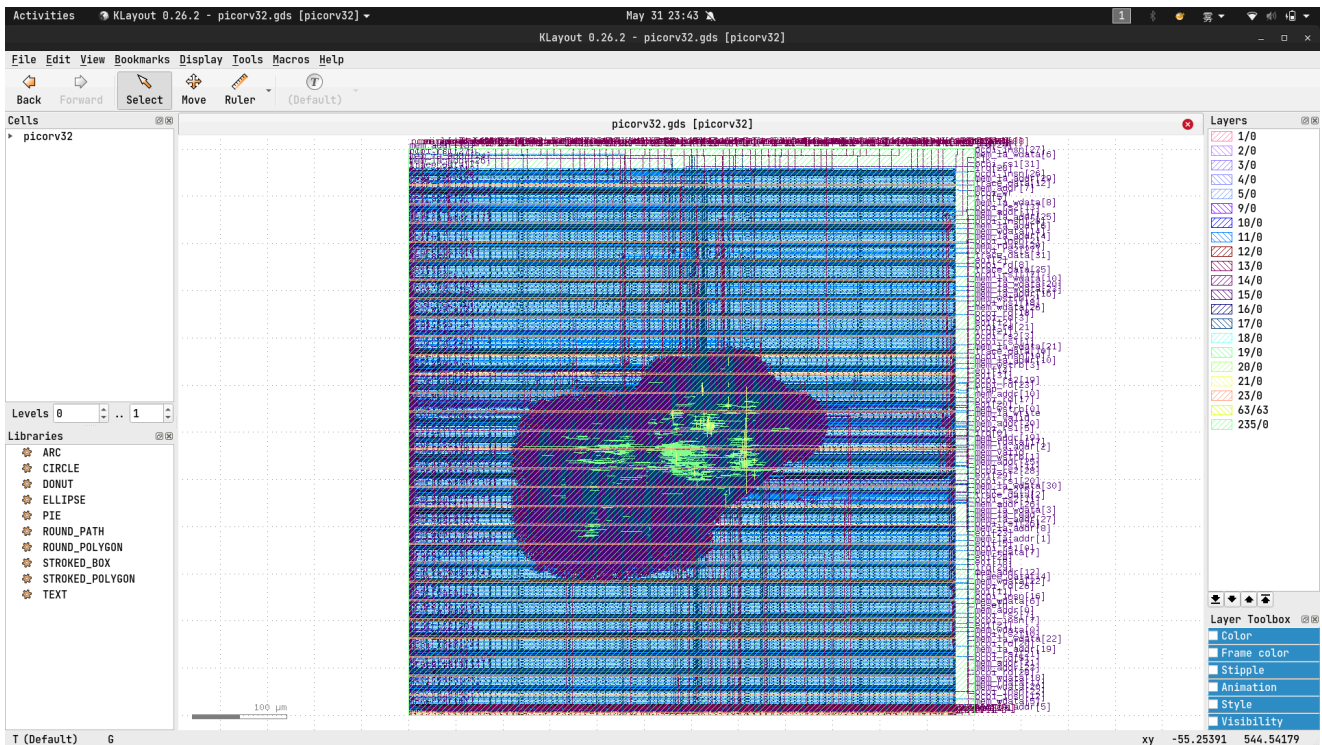
```
klayoutInsertLef.py : Finished
```

```
[INFO] Clearing cells...
```

```
[INFO] Merging GDS files...
```

```
/home/along/Desktop/iFlow/foundry/nangate45/gds/NangateOpenCellLibrary.gds
```

```
[INFO] Copying toplevel cell 'picorv32'
```



```
set DIE_AREA "0 0 150.2 150.2"
set CORE_AREA "1.08 1.08 140.12 140.12"
```

```
[INFO ODB-0134] Finished DEF file:
```

```
/home/along/Desktop/iFlow/result/picorv32.pdn.openroad_1.2.0.nangate45.HD.TYP.default/picorv32.def
```

```
[INFO GPL-0002] DBU: 2000
```

```
[INFO GPL-0003] SiteSize: 380 2800
```

```
[INFO GPL-0004] CoreAreaLxLy: 1900 0
```

```
[INFO GPL-0005] CoreAreaUxUy: 280060 280000
```

```
[INFO GPL-0006] NumInstances: 14145
```

```
[INFO GPL-0007] NumPlaceInstances: 13894
```

```
[INFO GPL-0008] NumFixedInstances: 251
```

```
[INFO GPL-0009] NumDummyInstances: 0
```

```
[INFO GPL-0010] NumNets: 15610
```

```
[INFO GPL-0011] NumPins: 45356
```



```

[INFO GPL-0012] DieAreaLxLy: 0 0
[INFO GPL-0013] DieAreaUxUy: 300400 300400
[INFO GPL-0014] CoreAreaLxLy: 1900 0
[INFO GPL-0015] CoreAreaUxUy: 280060 280000
[INFO GPL-0016] CoreArea: 77884800000
[INFO GPL-0017] NonPlaceInstsArea: 267064000
[INFO GPL-0018] PlaceInstsArea: 90882624000
[INFO GPL-0019] Util(%): 117.09
[INFO GPL-0020] StdInstsArea: 90882624000
[INFO GPL-0021] MacroInstsArea: 0
[ERROR GPL-0301] Utilization exceeds 100%.

```

失败

```

set DIE_AREA          "0 0 300.2 300.2"
set CORE_AREA         "1.08 1.08 290.12 290.12"

```

```

-----
FR_MASTERSLICE      0
    metal1      52706
    metal2      58611
    metal3       7523
    metal4      1378
    metal5        81
    metal6         0
    metal7         0
    metal8         0
    metal9         0
-----
                        120299

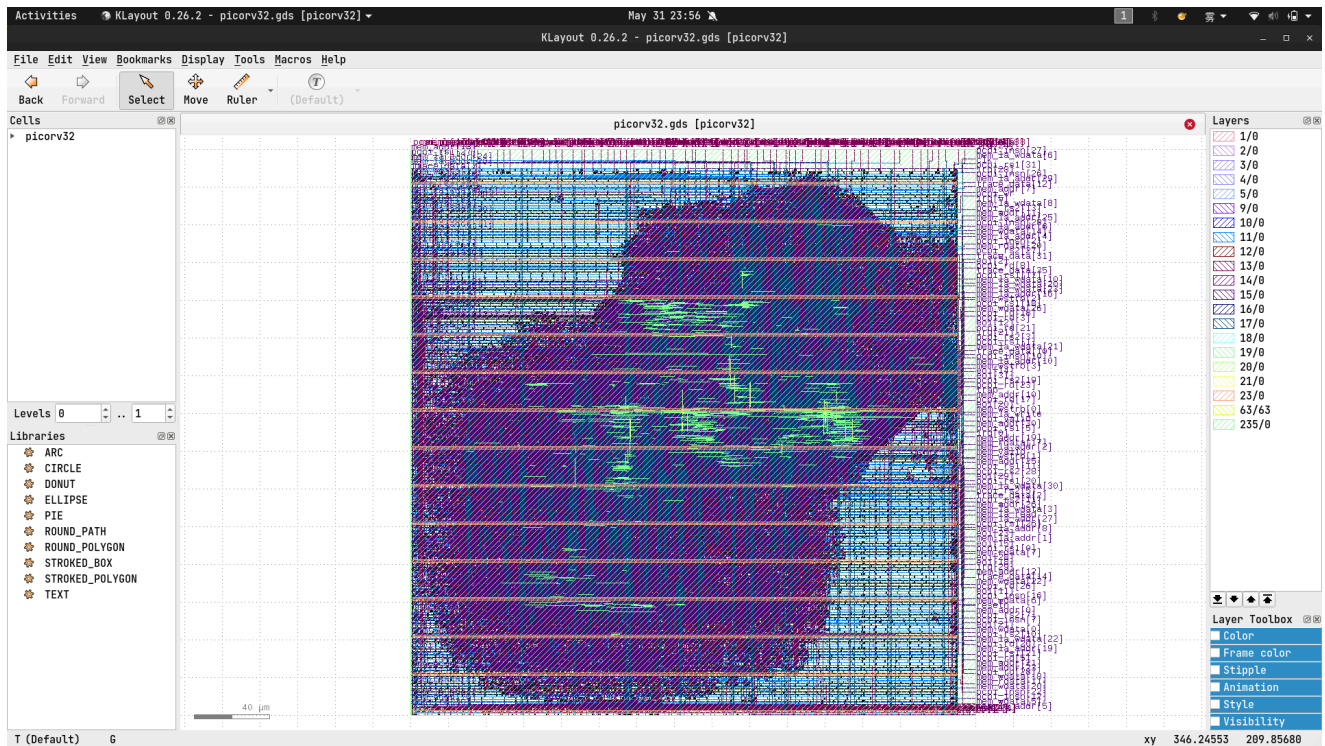
```

```

[INFO DRT-0172] cpu time = 01:09:09, elapsed time = 00:10:22, memory =
1267.77 (MB), peak = 1267.77 (MB)

```

比较合适的一个参数结果



提出建议

需要用户手动填写需要编译的文件，非常不方便。

有简易的文本处理函数能够完成该功能（当然有前提需要路径中没有无关的文件,也可以类似.gitignore这样的思路让用户把不需要参与编译的文件显示声明在.iflowignore中）思路如下，用户将顶层设计模块放在文件夹中，脚本自动递归的读取该文件夹中的编译文件，读取.iflowignore来去掉不参与编译的文件。